

LM-Vision: Persistent Database & Real-Time Sync Guide

Standalone Implementation Guide for Antigravity: SQLite Backend Persistence & React Global State Synchronization

Legal Metrology Packaged Commodities Compliance System (SIH 2026)

Referenced from [SIH2026 Pitch - Thunderhawks](#)

1. System Objective & Architecture

This guide specifies how to implement permanent data storage and real-time state synchronization across the LM-Vision system so that:

- **Zero Data Loss:** All scans, inspection reports, statutory violations, and rule amendments are committed to a permanent SQLite database on disk (`Backend/compliance.db`). Data persists when the browser or server closes.
- **Instant Cross-Component Reflection:** When an inspector records or overrides an inspection, the Authority Dashboard, Analytics KPI cards, and Audit Log tables update immediately across all screens without requiring a manual browser refresh.
- **Offline Resilience:** If the network or tunnel disconnects, records are safely cached in `localStorage` and synchronized once connectivity resumes.

2. Backend Implementation (FastAPI + SQLite + SQLAlchemy)

2.1 Database Setup & Models (Backend/database.py)

Create or update `Backend/database.py`:
from sqlalchemy import create_engine, Column, Integer, String, Float, Boolean, DateTime, Text

from sqlalchemy.ext.declarative import declarative_base

from sqlalchemy.orm import sessionmaker

```
import datetime
```

```
DATABASE_URL = "sqlite:///./compliance.db"
```

```
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":  
False})
```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```
class InspectionRecord(Base):
```

```
    __tablename__ = "inspections"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    commodity_name = Column(String, default="Packaged Commodity")
```

```
    dietary_type = Column(String, default="VEG") # VEG, NON_VEG, NON_FOOD
```

```
    officer_name = Column(String, default="Field Inspector")
```

```
    district_zone = Column(String, default="Chennai South")
```

```
    compliance_score = Column(Float, default=0.0)
```

```
    status = Column(String, default="COMPLIANT") # COMPLIANT or VIOLATION
```

```
    violations_count = Column(Integer, default=0)
```

```
    inspector_notes = Column(Text, nullable=True)
```

```
    raw_results_json = Column(Text, nullable=True) # Serialized JSON array of  
rules & OCR text
```

```

signature_url = Column(Text, nullable=True)    # Base64 digital signature

image_url = Column(String, nullable=True)

timestamp = Column(DateTime, default=datetime.datetime.utcnow)

class RuleRecord(Base):

    __tablename__ = "statutory_rules"

    id = Column(String, primary_key=True)

    code = Column(String, nullable=False)

    name = Column(String, nullable=False)

    mandatory = Column(Boolean, default=True)

    category = Column(String, default="ALL")    # ALL, FOOD_ONLY

    regex_pattern = Column(Text, nullable=True)

    description = Column(Text, nullable=True)

# Auto-create tables on startup

Base.metadata.create_all(bind=engine)

```

2.2 API Endpoints (Backend/main.py)

Add or update these endpoints in `Backend/main.py` to support persistent storage and analytics aggregation:

```

import json

from fastapi import FastAPI, Depends, HTTPException

from fastapi.middleware.cors import CORSMiddleware

```

```
from sqlalchemy.orm import Session

from database import SessionLocal, InspectionRecord, RuleRecord

app = FastAPI(title="LM-Vision Backend API")

app.add_middleware(

    CORSMiddleware,

    allow_origin_regex=r"https?://.*",

    allow_credentials=True,

    allow_methods=["*"],

    allow_headers=["*"],

)

def get_db():

    db = SessionLocal()

    try:

        yield db

    finally:

        db.close()

# 1. Record an Inspection (Persists to SQLite)

@app.post("/api/inspections/submit")
```

```
def submit_inspection(payload: dict, db: Session = Depends(get_db)):

    violations = payload.get("violations", [])

    results = payload.get("results", [])

    passed_count = sum(1 for r in results if r.get("status") == "PASS")

    total_count = len(results) if results else 1

    score = round((passed_count / total_count) * 100, 1)

    record = InspectionRecord(

        commodity_name=payload.get("commodityName", "Packaged Commodity"),

        dietary_type=payload.get("dietaryType", "VEG"),

        officer_name=payload.get("officerName", "Field Inspector"),

        district_zone=payload.get("districtZone", "General Zone"),

        compliance_score=score,

        status="VIOLATION" if len(violations) > 0 else "COMPLIANT",

        violations_count=len(violations),

        inspector_notes=payload.get("inspectorNotes", ""),

        raw_results_json=json.dumps(results),

        signature_url=payload.get("signatureUrl", None),

        image_url=payload.get("imageUrl", "")
```

```
)
```

```
db.add(record)
```

```
db.commit()
```

```
db.refresh(record)
```

```
return {"status": "success", "id": record.id, "score": score}
```

2. Retrieve All Saved Inspections

```
@app.get("/api/inspections")
```

```
def get_inspections(db: Session = Depends(get_db)):
```

```
    records =
```

```
    db.query(InspectionRecord).order_by(InspectionRecord.timestamp.desc()).all()
```

```
    output = []
```

```
    for r in records:
```

```
        output.append({
```

```
            "id": r.id,
```

```
            "commodityName": r.commodity_name,
```

```
            "dietaryType": r.dietary_type,
```

```
            "officerName": r.officer_name,
```

```
            "districtZone": r.district_zone,
```

```
            "complianceScore": r.compliance_score,
```

```

    "status": r.status,

    "violationsCount": r.violations_count,

    "inspectorNotes": r.inspector_notes,

    "results": json.loads(r.raw_results_json) if r.raw_results_json else [],

    "timestamp": r.timestamp.isoformat()

})

```

return output

3. Dynamic Aggregated Analytics

```
@app.get("/api/analytics/overview")
```

```
def get_analytics(db: Session = Depends(get_db)):
```

```
    total = db.query(InspectionRecord).count()
```

```
    violations = db.query(InspectionRecord).filter(InspectionRecord.status ==
"VIOLATION").count()
```

```
    compliant = total - violations
```

```
    rate = round((compliant / total * 100), 1) if total > 0 else 100.0
```

```
    veg_count = db.query(InspectionRecord).filter(InspectionRecord.dietary_type
== "VEG").count()
```

```
    non_veg_count =
db.query(InspectionRecord).filter(InspectionRecord.dietary_type ==
"NON_VEG").count()
```

```
return {  
  
  "totalScans": total,  
  
  "complianceRate": rate,  
  
  "noticesIssued": violations,  
  
  "dietaryBreakdown": {  
  
    "veg": veg_count,  
  
    "nonVeg": non_veg_count,  
  
    "nonFood": total - (veg_count + non_veg_count)  
  
  }  
  
}
```

3. Frontend Global Synchronization Layer

3.1 Global State Provider

([sih-project/src/context/ComplianceContext.jsx](#))

Create [sih-project/src/context/ComplianceContext.jsx](#):
`import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';`

```
import apiClient, { saveOfflineScan } from '../services/api';
```

```
const ComplianceContext = createContext();
```

```
export function ComplianceProvider({ children }) {
```

```
  const [inspections, setInspections] = useState([]);
```



```
const [analytics, setAnalytics] = useState({

  totalScans: 0,

  complianceRate: 100,

  noticesIssued: 0,

  dietaryBreakdown: { veg: 0, nonVeg: 0, nonFood: 0 }

});

const [loading, setLoading] = useState(false);

const refreshData = useCallback(async () => {

  try {

    setLoading(true);

    const [inspectionsRes, analyticsRes] = await Promise.all([

      apiClient.get('/inspections'),

      apiClient.get('/analytics/overview')

    ]);

    if (inspectionsRes.data) {

      setInspections(inspectionsRes.data);

      localStorage.setItem('cached_inspections',
JSON.stringify(inspectionsRes.data));

    }

  }

});
```

```
    if (analyticsRes.data) {  
  
        setAnalytics(analyticsRes.data);  
  
        localStorage.setItem('cached_analytics', JSON.stringify(analyticsRes.data));  
  
    }  
  
    } catch (err) {  
  
        console.warn('Backend offline, loading cached data from local storage:', err);  
  
        const cachedInspections = localStorage.getItem('cached_inspections');  
  
        const cachedAnalytics = localStorage.getItem('cached_analytics');  
  
        if (cachedInspections) setInspections(JSON.parse(cachedInspections));  
  
        if (cachedAnalytics) setAnalytics(JSON.parse(cachedAnalytics));  
  
    } finally {  
  
        setLoading(false);  
  
    }  
  
    }, []);  
  
    useEffect(() => {  
  
        refreshData();  
  
    }, [refreshData]);  
  
    const recordNewInspection = async (reportPayload) => {  
  
        try {
```

```

    await apiClient.post('/inspections/submit', reportPayload);

    await refreshData();

  } catch (err) {

    saveOfflineScan(reportPayload);

    const optimisticItem = {

      id: Date.now(),

      ...reportPayload,

      timestamp: new Date().toISOString(),

      status: reportPayload.violations?.length > 0 ? 'VIOLATION' : 'COMPLIANT'

    };

    setInspections(prev => [optimisticItem, ...prev]);

  }

};

return (

  <ComplianceContext.Provider

    value={{

      inspections,

      analytics,

```

```

        loading,

        refreshData,

        recordNewInspection
    }}

    >

    {children}

</ComplianceContext.Provider>

);

}

export const useCompliance = () => {

    const context = useContext(ComplianceContext);

    if (!context) {

        throw new Error('useCompliance must be used within a ComplianceProvider');

    }

    return context;

};

```

3.2 Wrap the Root Application (src/App.jsx)

Wrap `App.jsx` with `<ComplianceProvider>`.

3.3 Connect Inspector Submission (src/pages/InspectorPortal.jsx)

Update `handleConfirmSubmit` in `InspectorPortal.jsx` to use `recordNewInspection`.

3.4 Connect Authority Dashboard & Real-Time Metrics (src/pages/AuthorityPortal.jsx)

Update `AuthorityPortal.jsx` to consume live analytics from the context.

3.5 Persistent Audit Log Table (src/components/authority/AuditLogTable.jsx)

ID	Commodity	Classification	Zone	Score	Status	Timestamp
#1024	Packaged Commodity		Chennai South	95.0%	COMPLIANT	2026-05-12
#1023	Dairy Product		Madurai	40.0%	VIOLATION	2026-05-11
#1022	Meat Snacks		Salem	100%	COMPLIANT	2026-05-11

4. Prompt to Give Antigravity

Copy and paste this instruction to Antigravity:

"Please use this [Persistent Database & Real-Time Sync Guide](#) to add permanent SQLite database storage and global React state synchronization to my project:

1. In `Backend/`: Add `database.py` with SQLAlchemy SQLite setup (`compliance.db`) and update `main.py` with the `/api/inspections/submit`, `/api/inspections`, and `/api/analytics/overview` endpoints.
2. In `sih-project/src/`: Create `context/ComplianceContext.jsx` and wrap `App.jsx` with `<ComplianceProvider>`.
3. Update `InspectorPortal.jsx` so submitting inspection notices records directly to the database via `recordNewInspection()`.

4. Update `AuthorityPortal.jsx` and add `components/authority/AuditLogTable.jsx` to render live data from the database.
5. Ensure all data persists permanently when closing the app, and that new scans immediately update the counts and tables across all screens."